

Pertsonaia sortu

Atal honetan, **pertsonaia** bat sortuko da. Proiektuan erabiliko den pertsonaia **bola** bat izango da, oztopoak saihesten aurrera egingo duena. Bola sortzeko, hierarkian eskuineko botoia erabiliz "**3D Object**" aukeran "**Sphere**" hautatu behar da. Honek eszena batean esfera bat sortuko du, eta izena "**player**" izango da.

Pertsonaiaren kokapena

Lehenik eta behin, "**Transform**" osagaia berrezarri behar da, Unity-n objektu berri bat sortzean ausazko kokapen batean agertzen baita. Horretarako, "Transform" osagaien eskuineko botoiaz klik egin eta "**Reset**" aukera hautatu behar da. Honek posizioa (0,0,0), biraketa (0,0,0) eta eskala (1,1,1) izango ditu, eta esfera guztiz erdigunean kokatuko da.

Pertsonaiari biraketak eta fisikak gehitzea beharrezkoa da. Horretarako, "**Inspector**" atalean, "**Add Component**" aukera erabiliz "**Rigidbody**" izeneko osagaia gehituko zaio. Osagai honek objektuaren fisikak kudeatzen ditu, esaterako, grabitatearen eragina. Jolasa exekutatzean, ikus daiteke esfera beherantz erortzen dela.

Lurra Sortzea

Jarraitzeko, solido bat sortuko da bolari oinarri bat jartzeko "**3D Object**" aukeran "**Cube**" hautatu behar da, eta izena "**ground**" izango da, lurra balitz moduan jardungo duena. Ondoren, "**Transform**" osagaia berriro berrezarri eta eskalatu behar da lur itxura emateko (adibidez, X ardatzean 10 eta Z ardatzean 50). Jokalarien bola lurraren gainetik ondo kokatu arte igo behar da, eta lurra behar bezala mugitu daiteke.

Koloreak edo testurak Gehitzea

Bola eta lurra bereizteko, objektuei koloreak gehitu behar zaizkie. Horretarako, "**Materials**" izeneko karpeta sortu eta bertan bi material berri sortuko dira: "**Player Material**" jokalarientzat eta "**Ground Material**" lurrarentzat. Material horiei kolore desberdinak aplikatuko zaizkie, eta materiala objektuarekin erlazionatzeko, materiala objektura arrastatu behar da. Kolorea eta beste ezaugarri batzuk (adibidez, "**Smoothness**") egokitu daitezke.

Jokalariaren Mugimendua Programatzea

Jokalariaren mugimendua kudeatzeko, C#-ko script bat sortu behar da, eta izena "**ScriptJokalaria**" izango du. Script hau sortzeko, "**New Script**" aukera erabiliko da, eta ondoren editatu ahal izateko, "**Edit Script**" aukera hautatu behar da. Script-ak "**Start**" eta "**Update**" funtzioak lehenespenez sortzen ditu:

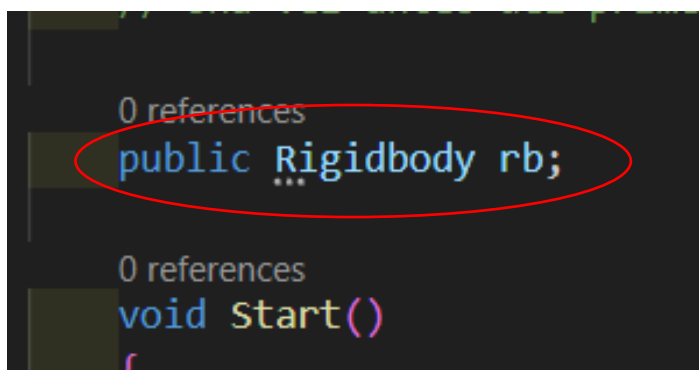
Jokalariaren ezaugarrien bukaeran script-a azalduko da, eskumako botoiari emon eta edit script aukeratu. Visual studio zabaldu beharko zan eta bertan start eta update agertzen dira. Hau beti defektuz sortzen du.

```
void Start()
{
    Debug.Log("Kaixo");
}

void Update()
{
    Debug.Log("Update");
}
```

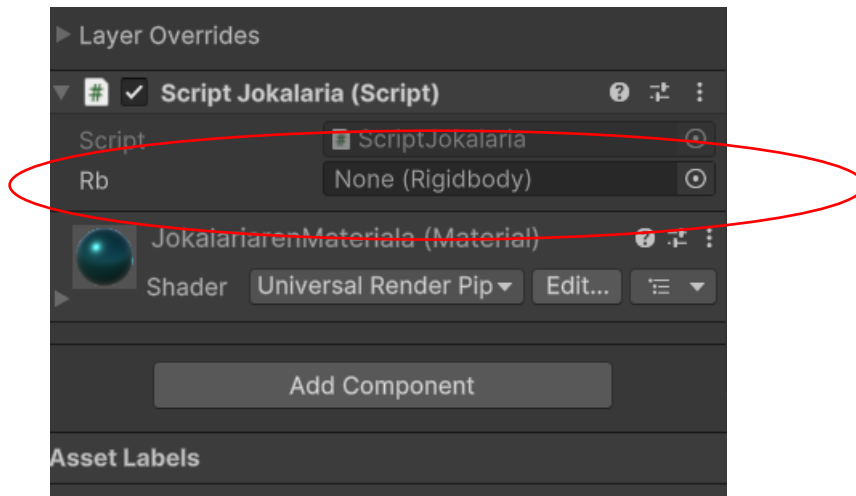
Script hau gordetzeko, "**Ctrl + S**" sakatu behar da eta Unityra itzuli. Ondoren, "**Rigidbody**" osagaia script-ean gehituko da objektua mugitzeko. Sortutako mezuak **Console**-n ikusiko dira. Laburtuta ikusi ahal izateko **Collapse**-ri emon.

Script honetan **Rigidbody** osagaia baliatuko da objektua mugitzeko. Beraz **script**-etik konponentea gehitu beharko da.

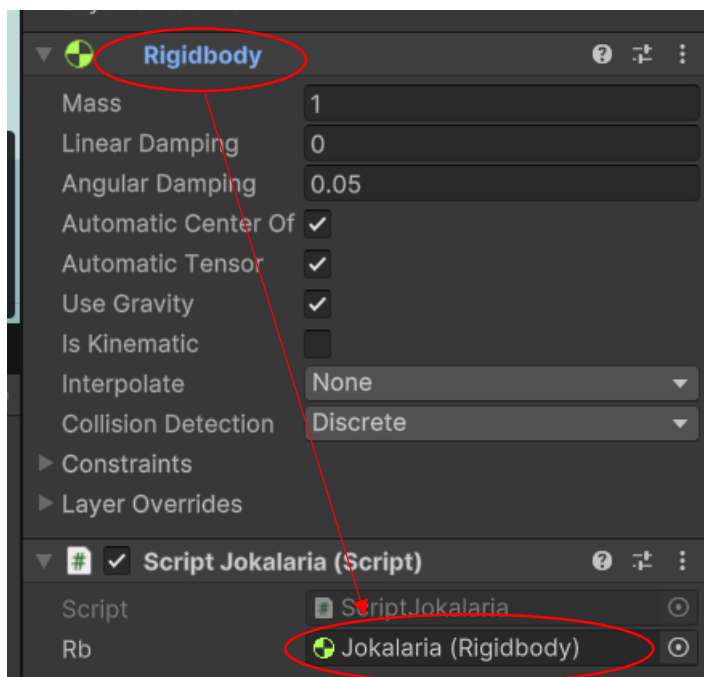


The screenshot shows the Unity Inspector window with a script component selected. The line `public Rigidbody rb;` is highlighted with a red oval. Above it, the text "0 references" is visible. Below it, the text "0 references" is also visible, followed by the `void Start()` method signature.

Sortutako aldagai berria Unity-n azalduko da. Hasiera baten hutsik egongo da, egoera hori aldatzeko RigidBody aldagaia arrastratu beharko da



Gure aldagai berria hutsik dau beraz RB aldagaia None hori aldatu beharko jako RigidBody aldagaia eramanez.



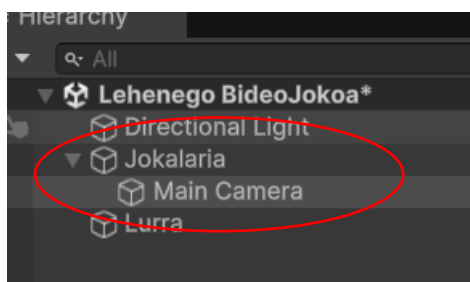
Mugimenduaren Parametroak

Jokalariaren mugimenduaren abiadura definitzeko, "Abiadura" izeneko aldagaia erabiliko da. "AddForce" metodoa erabiliko da indarra gehitzeko, eta mugimendua Z ardatzean egingo da (Kontutan eduki behar da ardatz desberdinean izen daitekeela). Ordenagailu desberdinen arteko desberdintasunak berdintzeko, "Time.deltaTime" biderkatuko da, eta horrek mugimendua uniforme izatea bermatuko du.

```
public Rigidbody rb;
public float abiadura;
// Mensaje de Unity | 0 referencias
void Start()
{
    Debug.Log("Kaixo");
}

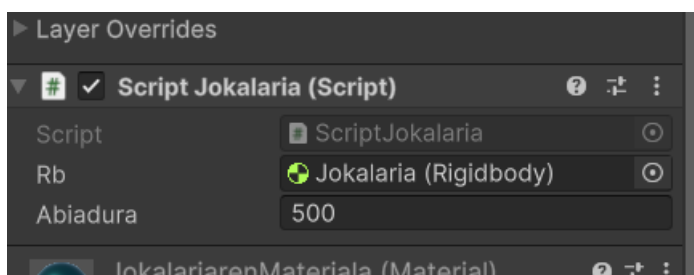
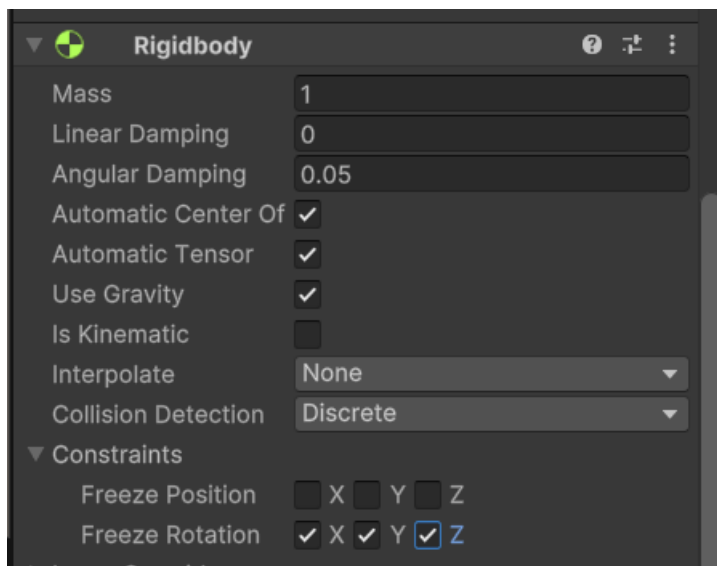
// Mensaje de Unity | 0 referencias
void Update()
{
    //Debug.Log("Update");
    rb.AddForce(new Vector3(0, 0, abiadura) * Time.deltaTime);
}
```

Hau probatuz gero ikusten da bola mugitzen dela baina kamara geldi dagoela. Jokalariaren mugimenduarekin batera kamara jarraitzea nahi bada, kamara jokalariaren menpeko (child) objektu bihurtuko behar da. Kameraren posizioa eta biraketa egokituz, lurraren gaineko ikuspegi egokia lortuko da honekin.



Biraketak Blokeatzea

Azkenik, biraketak blokeatuko dira jokalariaren "Rigidbody" osagaian, eta mugimendua hobeto kontrolatzeko "Constraints" ataleko biraketak (X, Y eta Z) blokeatuko dira. Jokatzeko orduan, abiadura eta mugikortasuna egokitu daitezke balio desberdinak erabiliz.



Teklen bidez Mugimendua

Jokalaria ezkerrera eta eskuinera mugitzeko, adibidez, "A" eta "D" teklak erabiliko dira. Horretarako, baldintzak erabiliko dira, eta tekla bakoitzari dagokion indarra gehituko zaio. Ondorioz, jokalaria kontrol errazagoa izango du mugimenduan.

```
if (Input.GetKey(KeyCode.A))  
{  
}
```

Ezkerrera eta Eskuinerako Mugimendua

Jokalaria ezkerrerantz mugitzeko, "A" tekla erabiliko da, eta eskuinerantz mugitzeko, "D" tekla. Horretarako, "Mugitu" aldagaia sortuko dugu. Mugimendua indar gehigarriekin kudeatuko da "AddForce" metodoaren bidez. Adibidez, ezkerreko mugimendua egiteko, indarra X ardatzean negatiboa izango da, eta eskuineko mugimenduan, positiboa. Baldintzen bidez tekla bakoitza egiaztatuko da, eta dagokion indarra aplikatuko zaio jokalariai.



Horrela, kontrol optimizatua lortuko da, eta jokoaren esperientzia hobetuko da.

```
//Ezkerrea mugitzeko
if (Input.GetKey(KeyCode.A))
{
    rb.AddForce(new Vector3(-mugitu, 0, 0) * Time.deltaTime);
}
//Eskumara mugitzeko
if (Input.GetKey(KeyCode.D))
{
    rb.AddForce(new Vector3(mugitu, 0, 0) * Time.deltaTime);
}
```